Experiment 5
Naive Bayes Classification

Aim:

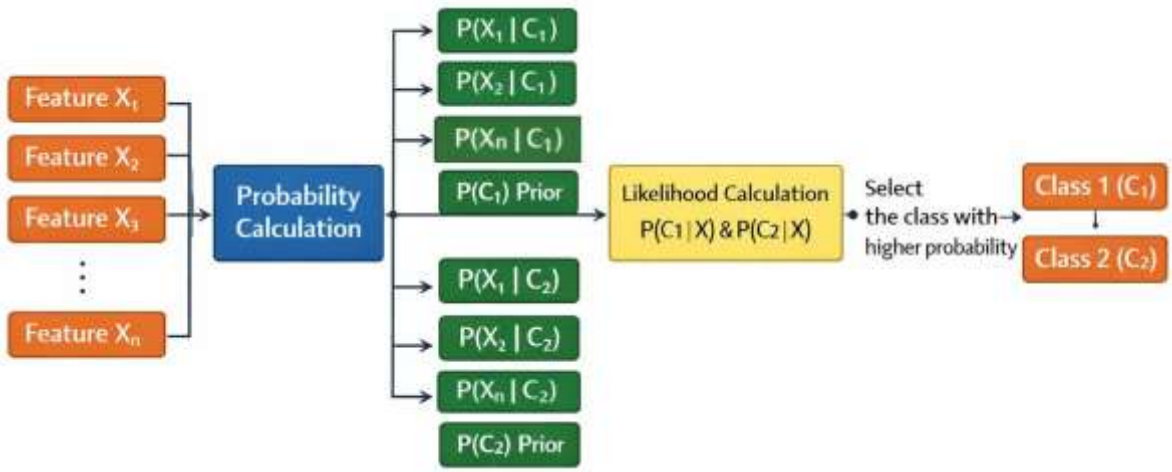
To implement the Naïve Bayes algorithm for predicting class labels and to evaluate its performance

Theory:

Naive Bayes is a supervised probabilistic learning algorithm widely used for text classification tasks such as spam and ham message detection. The model is based on Bayes' theorem, which estimates the posterior probability of a class given observed featur. A key assumption of Naive Bayes is that features are conditionally independent given the class label. While this assumption does not fully hold for natural language, the classifier remains effective due to the high dimensionality and sparse distribution of textual features.
Bayes' theorem is defined as:

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

where $P(C \mid X)$represents the probability of a message belonging to class C(spam or ham) given the feature set X, $P(X \mid C)$is the likelihood of observing the features under class C, P(X) is the marginal likelihood of the features, and $P(C)$denotes the prior probability of the class. The feature-wise conditional probabilities and class priors are computed to select the class with the highest posterior probability is shown in the figure below:



In text classification, messages are modelled using the bag-of-words representation, where word order is ignored and word occurrences form the basis of feature extraction. To improve feature quality and reduce redundancy, Natural Language Processing (NLP) techniques are applied. These include text normalization, removal of stop words, and reduction of words to their base forms using stemming and lemmatization. Part-of-Speech (POS) tagging supports lemmatization by enabling accurate identification of grammatical roles such as nouns and verbs, thereby preserving semantic correctness while reducing dimensionality.

Textual data is transformed into numerical vectors using Term Frequency–Inverse Document Frequency (TF-IDF) weighting. TF-IDF enhances the importance of words that are frequent within a document but infrequent across the entire corpus, thus improving the discriminative ability of the feature space.
For classification, the Multinomial Naive Bayes algorithm is employed, as it is well suited for word-based features and TF-IDF representations. Model effectiveness is assessed using standard metrics such as accuracy, precision, recall, confusion matrix, and classification report.

Merits of Naïve Bayes:

- Naive Bayes is suitable when the number of features is very large, such as in text classification problems with thousands of words.
- Naive Bayes is suitable when fast training and prediction are required because of its low computational complexity.
- Even when only a small amount of training data is available, Naive Bayes can still give acceptable results.
- When the dataset contains unnecessary or extra features, Naive Bayes can handle them without much loss in performance.

Demerits of Naïve Bayes:

- When features are highly correlated or dependent, Naive Bayes should not be used because the independence assumption is violated.
- When the problem is complex and non-linear, Naive Bayes is not suitable since it cannot model complex patterns.

Overall, the Naive Bayes classifier, combined with NLP-based feature normalization and TF-IDF representation, provides an efficient and robust solution for spam and ham message classification.

Algorithm:

Step 1: Apply **Bayes' Theorem**:

- $P(Class|Features) = P(Features|Class) \times P(Class) / P(Features)$

Step 2: Apply **Naive Independence Assumption**:

- $P(x_1,x_2,...,x_n|Class) = P(x_1|Class) \times P(x_2|Class) \times ... \times P(x_n|Class)$

Step 3: During Training, calculate from data:

- **Prior Probability:**
  - $P(Class=c) = (\text{Count of samples in class } c) / (\text{Total samples})$
- **Conditional Probability (for each feature):**
  - **For categorical features:**
    - $P(x_j=v|Class=c) = (\text{Count of class } c \text{ samples where feature } j = v) / (\text{Count of class } c \text{ samples})$
  - **For continuous features (Gaussian Naive Bayes):**
    - Calculate mean (μ) and variance (σ^2) of feature j for each class
    - $P(x_j|Class=c) = (1/\sqrt{(2\pi\sigma^2)}) \times \exp(-(x_j-\mu)^2/(2\sigma^2))$

Step 4: Add **Laplace Smoothing** (to handle zero probabilities):

- $P(x_j=v|Class=c) = (\text{Count} + 1) / (\text{Total} + \text{Number of unique values})$

Step 5: For **Prediction**:

- For each class c:
  - Calculate: $Score(c) = P(Class=c) \times \prod P(x_j|Class=c)$
- Use log to avoid underflow:
  - $Log_Score(c) = \log(P(Class=c)) + \Sigma\log(P(x_j|Class=c))$
- Predict class with **HIGHEST** score

Naive Bayes - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Step 1: Importing Libraries

Importing necessary libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
from IPython.display import HTML, display
print("Libraries Imported")
```

Output:

```
Libraries Imported
```

Step 2: Loading Dataset

Read the Spam SMS dataset

```
data = pd.read_csv('spam_detection.csv', encoding='latin1', header=0, usecols=[0,1],
    engine="python")
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Step 3: Data Analysis

Display first 5 rows

```
data.head()
```

Output:

	Category	Message
0	spam	Get your garden ready for summer with a free selection of summer bulbs and see..
1	spam	This is your chance to be on a reality fantasy show. Call now at 08707509020. ..
2	ham	Love is not a decision; it is a feeling. If we could choose whom to love, life..
3	ham	Good friends care for each other, close friends understand each other, and tru..
4	spam	Get your garden ready for summer with a free selection of summer bulbs and see..

Display Last 5 rows

```
data.tail()
```

Output:

	Category	Message
1737	spam	Do you want 750 anytime any network mins 150 text and a NEW VIDEO phone for only..
1738	ham	Thanx...
1739	spam	88800 and 89034 are premium phone services call 08718711108
1740	spam	Today's Offer! Claim ur £150 worth of discount vouchers! Text YES to 85023 now! ..
1741	ham	am up to my eyes in philosophy

Displays summary of dataset, including column names, data types, and non-null counts.

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1742 entries, 0 to 1741
Data columns (total 2 columns):
#   Column    Non-Null Count  Dtype
---  ------    --------------  -----
0   Category  1742 non-null   object
1   Message   1742 non-null   object
dtypes: object(2)
memory usage: 27.3+ KB
```

Checks the dataset for missing(NULL) values in each column

```
data.isnull().sum()
```

Output:

```
Category    0
Message     0
dtype: int64
```

Shows the size of the dataset

```
data.shape
```

Output:

```
(1742, 2)
```

Counts the frequency of each unique category

```
data['Category'].value_counts()
```

Output:

Category	Count
Ham	1003
Spam	739

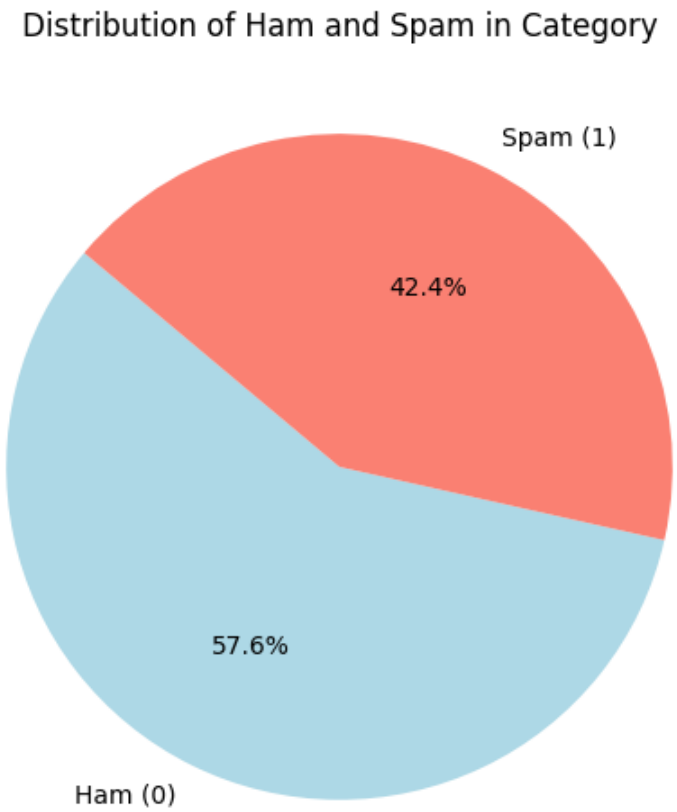
Output:
```
dtype: int64
```

Step 4: Data Preprocessing

 # Visualizes the distribution of spam and ham messages in the dataset

```
category_counts = data['Category'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(category_counts, labels=['Ham (0)', 'Spam (1)'],
    autopct='%1.1f%%', startangle=140, colors=['lightblue', 'salmon'])
plt.title('Distribution of Ham and Spam in Category');
plt.show()
```

Output:
```
Distribution of Ham and Spam in Category
```



 # Sets up stemming and stop-word removal tools for text preprocessing.

```
stemmer = PorterStemmer()
stop_words = set(ENGLISH_STOP_WORDS)
print("Initialized Porter Stemmer and English stopword list.")
```

Output:
```
Initialized Porter Stemmer and English stopword list.
```

 # Performs basic text cleaning and normalisation

```
def preprocess_text(s):
    if pd.isnull(s): return ""
    s = s.lower(); s = re.sub(r'http\S+|www\.\S+', ' ', s); s = re.sub(r'[^a-z0-9\s]', '
    ', s)
    s = re.sub(r'\s+', ' ', s).strip(); return s
print("Text Cleaned")
```

Output:

```
Text Cleaned
```

Step 5: Model Training

 # Encodes spam labels and splits the data into stratified training and testing sets.

```
data['label'] = (data['Category'].str.lower().str.strip() == 'spam').astype(int)
X_train, X_test, y_train, y_test = train_test_split( data['Message'], data['label'],
    test_size=0.25, random_state=42, stratify=data['label'] )
print("Labels encoded and data split into training and testing sets.")
```

Output:

```
Labels encoded and data split into training and testing sets.
```

 # Converts text messages into TF-IDF feature vectors.

```
vectorizer = TfidfVectorizer(ngram_range=(1,3), min_df=2, max_features=50000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
print("Text data vectorized using TF-IDF.")
```

Output:

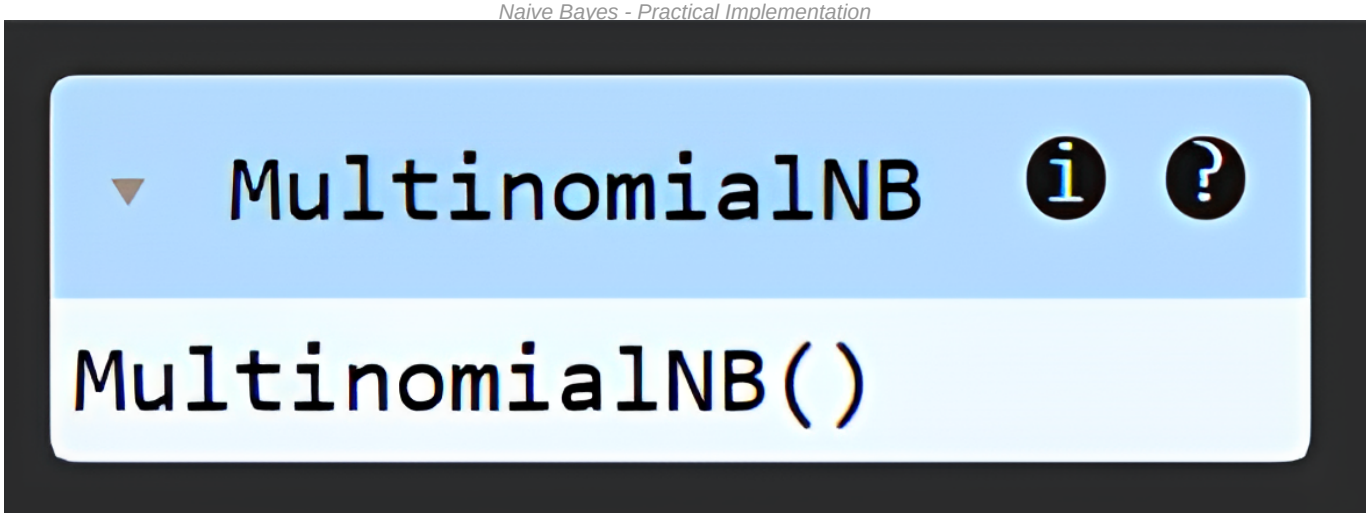
```
Text data vectorized using TF-IDF.
```

 # Trains a Multinomial Naive Bayes classifier on the TF-IDF features.

```
model=MultinomialNB()
model.fit(X_train_vec,y_train)
print("Model training completed.")
```

Output:

```
Model training completed.
```



Step 6: Model Evaluation

 # Predicts on test data and evaluates model

```
y_pred = model.predict(X_test_vec)
y_prob = model.predict_proba(X_test_vec)[:, 1]
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\\nClassification Report:\\n", classification_report(y_test, y_pred))
```

Output:

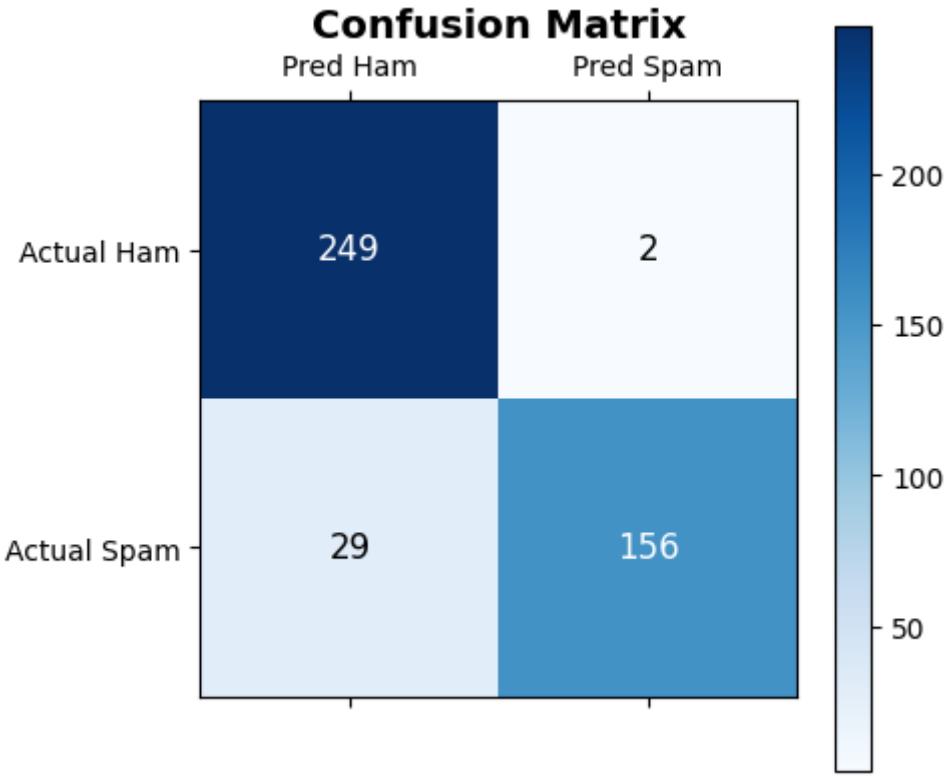
```
Accuracy: 0.9288990825688074
Classification Report:
precision    recall  f1-score   support
0(ham)       0.90      0.99      0.94       251
1(spam)       0.99      0.84      0.91       185
accuracy                           0.93       436
macro avg       0.94      0.92      0.93       436
weighted avg       0.93      0.93      0.93       436
```

 # Plots the confusion matrix with values to visualize correct and incorrect predictions

```
cm = confusion_matrix(y_test, y_pred); plt.matshow(cm, cmap='Blues')
for (i, j), val in np.ndenumerate(cm):
    plt.text(j, i, val, ha='center', va='center', color='white' if val > cm.max()/2 else
    'black', fontsize=12)
plt.xticks([0,1], ['Pred Ham','Pred Spam'])
plt.yticks([0,1], ['Actual Ham','Actual Spam']); plt.colorbar();
plt.show(); print("Confusion matrix displayed.")
```

Output:

```
Confusion matrix displayed.
```

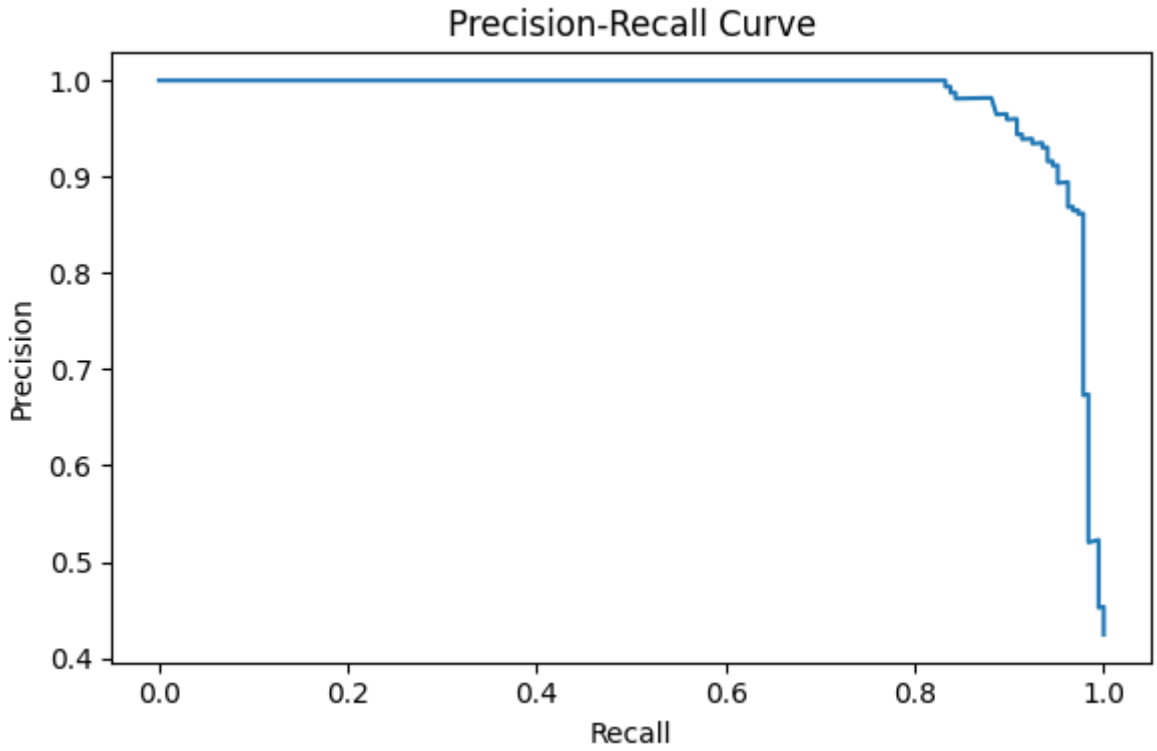


 # Plots the Precision-Recall curve for the model

```
precision, recall, _ = precision_recall_curve(y_test, y_prob)
plt.figure(figsize=(6,4)); plt.plot(recall, precision); plt.title('Precision-Recall
    Curve'); plt.xlabel('Recall');plt.ylabel('Precision'); plt.tight_layout(); plt.show()
print("Precision-Recall curve displayed.")
```

Output:

```
Precision-Recall curve displayed.
```



 # Highlights words based on their contribution to spam/ham prediction using color intensity

```
def color_text(words, contributions):
    max_val = max(abs(c) for c in contributions) or 1; html = ""
    for w, c in zip(words, contributions):
        intensity = int(255 * (abs(c) / max_val))
        if c > 0: color = f"rgb(255,{255-intensity},{255-intensity})"
        else: color = f"rgb({255-intensity},255,{255-intensity})"
        html += f"{w} "
    return html
print("Text contribution visualization function defined.")
```

Output:

```
Text contribution visualization function defined.
```

 # Computes and displays precision and recall of the model on test data

```
def show_precision_recall(model, X_test_vec, y_test):
    preds = model.predict(X_test_vec)
    precision = precision_score(y_test, preds) ;recall = recall_score(y_test, preds)
print("Precision and recall calculation function ready.")
```

Output:

```
Precision and recall calculation function ready.
```

 # Performs precision-recall evaluation and analyzes 'n' random test messages

```
def analyze_multiple(model, vectorizer, X_test, y_test, n=5):
    show_precision_recall(model, X_test_vec, y_test)
    for i in range(n):
        analyze_random_message(model, vectorizer, X_test, y_test)
print("Multiple message analysis function defined.")
```

Output:
```
Multiple message analysis function defined.
```

Runs evaluation and displays analysis of 5 random test messages

```
analyze_multiple(model, vectorizer, X_test, y_test, n=5)
print(" MODEL PERFORMANCE ON TEST DATA")
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
```

Output:
```
MODEL PERFORMANCE ON TEST DATA
Precision: 0.9873
Recall   : 0.8432
Choose one random test sample.
```

Output:

Category	Message
ham	It's getting me down just waiting around.
spam	Text and meet someone today. Reply with your name and age.
ham	I.ll give her once i have it. Plus she said grinule greet you whenever we speak
spam	Congratulations U can claim 2 VIP row A Tickets 2 C Blu in concert in November or Blu gi..
ham	thinking about you.

Output:
```
Message Analysis
Actual:
Predicted:
Spam Probability:
```

Interactive Features (Static View)

Interactive Feature: Evaluation Sample Analysis

Clicking an evaluation sample shows the model prediction and word influence analysis. Below are all 5 test samples with results:

#	Category	Message	Predicted	Spam Prob
1	ham	It's getting me down just waiting around.	ham	0.0408
2	spam	Text and meet someone today. Reply with your name and age.	spam	0.5772
3	ham	I'll give her once i have it. Plus she said greet you when..	ham	0.0322
4	spam	Congratulations U can claim 2 VIP Tickets to C Blu in conc..	spam	0.9309
5	ham	thinking about you.	ham	0.0670

Interactive Feature: Random Message Prediction

The simulation tests the model on random messages:

#	Message	Actual	Predicted
1	Working on the project now.	Ham	Ham
2	WINNER! You won a FREE gift!	Spam	Spam
3	Lunch at 1pm?	Ham	Ham